

整合

批次效应的本质：同一种细胞，在批次A和批次B中测，表达谱具有系统偏差，整体往某一方向**偏移**。

若要消除批次效应，或称对齐，即**让不同批次里同类型的细胞，在分析里视为同类**，则需要把偏移算出来再折回去，共三种常见方法：

- Harmony：
 - 把 细胞位置 = 真实位置 + 批次偏移 拆开，迭代估计并减去偏移
 - 机制
 - a. 看数据，估计偏移向量
 - b. 把偏移减去，矫正细胞位置
 - c. 看看有没有残留偏移，如果有则回到 1.，直到可忽略不计
- BBKNN
 - 不改坐标，改邻居图的“连接”。例如，批次A和批次B的T细胞因连上了而归为一群，即使坐标没变
 - 区别
 - 普通邻域图：每个细胞找最近的n个邻居，则批次A的T细胞连的都是批次A的
 - BBKNN：每个细胞找邻居时，**强制每个批次各贡献几个**，则批次A的T细胞能连到批次B的T细胞
- ingest
 - 不消除批次效应，只借标签。
 - 相当于不碰表达和坐标，只用**参考数据**的模型给**查询数据**贴标签
 - 机制
 - a. 参考数据建好（**预训练**）模型
 - a. PCA模型：坐标系
 - b. 邻居图：kNN查找树
 - c. UMAP：地图
 - b. 查询数据投影进PCA空间（**嵌入，前向传播**）
 - c. 在邻居图里找最近邻居，并贴参考标签（**嵌入最近邻分类头，作分类**）
 - d. 用参考的UMAP算查询数据的坐标（**用训练好恶毒可视化映射作投影**）

PBMC数据

使用PBMC，即Peripheral Blood Mononuclear Cells, 外周血单个核细胞作为数据。

接下来以两个小数据集为例：

- `adata_ref`：已完成注释的参考数据集，3k
- `adata`：需要查询标签和嵌入坐标的数据集，68k

由于一部分已经注释了，一部分还没有注释。因此

如果需要整合，两个数据集必须在同一组变量上作比较，通常是基因

```
# 取两个数据集的共同基因，其中 .intersection()代表取交集
var_names = adata_ref.var_names.intersection(adata.var_names)
adata_ref = adata_ref[:, var_names].copy()
adata = adata[:, var_names].copy()
# 让两个数据集只保留有交集的基因
```

ingest

1. 建模型

在参考数据上训练得到模型与图（如PCA、邻域图和UMAP）

```
sc.pp.pca(adata_ref)
sc.pp.neighbors(adata_ref)
sc.tl.umap(adata_ref)
```

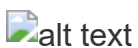
2. 映射

基于表示形式（如PCA坐标、UMAP坐标），将参考数据的注释和嵌入坐标从 `adata_ref` 映射到 `adata`。下以 PCA 为例（维度低，信息密），映射聚类标签和 UMAP 坐标

```
# 1. 将adata_ref的 louvain 列（注释信息）映射到 adata
sc.tl.ingest(adata, adata_ref, obs="louvain")

# 2. 统一adata_ref和adata的颜色，保存在.uns中
adata.uns["louvain_colors"] = adata_ref.uns["louvain_colors"]

# 画UMAP验证
sc.pl.umap(adata, color=["louvain", "bulk_labels"], wspace=0.5)
```



adata的ingest验证图

- 左：louvain
 - 用ingest映射来的标签louvain着色

- 分类粗，因为louvain分类本来就粗
 - 右：bulk_labels
 - 用查询数据自带的真值bulk_labels着色
 - 总体映射合理，但右侧的小集群里
 - 左：基本是棕色 + 橙色，为Monocytes
 - 右：基本是棕色 + 青色，为Monocytes + Dendritic
- 综上：**在adata中，Monocytes和Dendritic具有批次效应**

为了验证映射之后，两批数据是否融合在一起，有如下代码

```
# 创建一个adata_concat，两批数据纵向拼接，并新增一列，列名为batch，分别对应ref和new
adata_concat = anndata.concat([adata_ref, adata], label="batch", keys=["ref", "new"])

# 虽然拼接后louvain都有数据，但是类别顺序不同。因此要统一类别顺序和颜色
adata_concat.obs["louvain"] = (
    adata_concat.obs["louvain"].astype("category").cat.reorder_categories(adata_ref.obs["louvain"].cat.categories)
)

adata_concat.uns["louvain_colors"] = adata_ref.uns["louvain_colors"]
# 画图
sc.pl.umap(adata_concat, color=["batch", "louvain"])
```

 alt text

- 左图：
 - ref和new混在一起，没有说一边ref一边new
 - 说明映射/整合后，两批数据在同一UMAP里没有互相排挤
 - **结论**：无明显批次效应，技术差异小
- 右图：
 - 除了少数异常点外，每种细胞类型是一整片连续区域
 - **结论**：生物差异清晰，类型得以清晰分离
- **总结**：ingest 映射成功，因为两批数据既能够不按批次分开，又能按细胞类型分开

总结

1. 参考(adata_ref):
 - i. 基因对齐
 - ii. PCA
 - iii. 邻居图
 - iv. UMAP

2. 查询(adata):

- i. 投影进参考的 PCA 空间
- ii. 查参考的邻居树：快速找出离某adata最近的adata_ref是谁
- iii. 找最近的参考细胞借 louvain 标签
- iv. 用参考 UMAP 模型算自己的坐标

3. 画图

- i. adata 的 louvain(借的)vs bulk_labels(真值)
- ii. adata_concat 的 batch(看有无明显批次效应) vs louvain(看看细胞分类是否清晰)

BBKNN

将ref和new当做两个平等批次对称整合，抛弃了 参考/查询 的不对称设定：

```
sc.tl.pca(adata_concat)
```

```
sc.external.pp.bbknn(adata_concat, batch_key="batch")
```

```
sc.tl.umap(adata_concat)
```

```
sc.pl.umap(adata_concat, color=["batch", "louvain"])
```

 alt text

可见

- 由于巨核细胞(Megakaryocytes)只存在于ref中，所以BBKNN强制每个批次都找邻居，会导致adata无巨核细胞连，因此巨核细胞不成群，**没有保留巨核细胞聚类**
- 由于跨批次邻居强制拉近，得以让细胞混合更均一

Harmony

```
# ① 合并后重新 PCA
sc.tl.pca(adata_concat)

# ② Harmony 整合
sc.external.pp.harmony_integrate(adata_concat, key="batch")
# 产出:adata_concat.obsm["X_pca_harmony"]

# ③ 用 Harmony 坐标重新建邻居图 + UMAP
sc.pp.neighbors(adata_concat, use_rep="X_pca_harmony")
sc.tl.umap(adata_concat)

# ④ 画图对比
sc.pl.umap(adata_concat, color=["batch", "louvain"], wspace=0.5)
```

 alt text

胰腺数据

- 有四个批次，分别命名为"1", "2", "3", "4"
- 自带celltype真值列，其中细胞类型高度不平衡，其中alpha和beta占了大部分

1. 观察细胞类型以及数量

```
counts = adata_all.obs["celltype"].value_counts()

counts.to_frame()
```

2. 观察批次效应

```
sc.pp.pca(adata_all)
sc.pp.neighbors(adata_all)
sc.tl.umap(adata_all)

sc.pl.umap(adata_all, color=["batch", "celltype"], palette=sc.pl.palettes.vega_20_scanpy)
```

 alt text

- 左图：数据主要变异轴为批次
- 右图：类型和批次息息相关
- 以alpha, beta, ductal, delta细胞类型和batch 0, 1为例，发现batch 1 的四种细胞类型都相对于batch 0更加靠左，得知**同样的细胞类型只因批次不同而分开**，故存在明显的批次效应

BBKNN

```
# 进行BBKNN整合，并明确批次变量batch_key为adata_all的"batch"
sc.external.pp.bbknn(adata_all, batch_key="batch")

# 计算UMAP，得到adata_all.obsm["X_umap"]
sc.tl.umap(adata_all)

# 画2张图，分别按batch和celltype着色
sc.pl.umap(adata_all, color=["batch", "celltype"])
```

 alt text

- 左图
 - 批次未完全混匀
 - 但不代表整合失败，毕竟有组成差异
- 右图
 - 同类型细胞跨批次整合了
- 结论
 - **BBKNN整合成功，实现同类型跨批次连接**
 - 如果数据本身没有celltype列，则BBKNN不能指明细胞类型。如果仅batch 0具有celltype，则可以使用ingest来注释batch 1, 2, 3

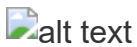
ingest

选择batch 0作为参考batch来训练模型并建立PCA、邻域图和UMAP

```
adata_ref = adata_all[adata_all.obs["batch"] == "0"].copy()

sc.pp.pca(adata_ref)
sc.pp.neighbors(adata_ref)
sc.tl.umap(adata_ref)

sc.pl.umap(adata_ref, color="celltype")
```



参考batch包含所有batch19种细胞的12种

再将另外三个作为查询batch，将标签(celltype)与嵌入坐标(X_pca, X_umap)从参考batch迭代映射到查询batch

```
# 切出查询batch，adatas里含有batch1,2,3，没有0
adatas = [adata_all[adata_all.obs["batch"] == i].copy() for i in ["1", "2", "3"]]

# 调高ingest日志详细度
sc.settings.verbosity = 2

# 进行逐个ingest
# enumerate(adatas, 1) = [(1, batch1数据), (2, batch2数据), (3, batch3数据)], 即给列表里每个元素配
for iadata, adata in enumerate(adatas, 1):
    # 备份真值
    adata.obs["celltype_orig"] = adata.obs["celltype"]
    # 进行ingest映射，并覆盖掉原celltype
    sc.tl.ingest(adata, adata_ref, obs="celltype")
```

现在每个查询batch都有了参考batch的注释，接下来就拼接并查看

```
# 合并参考和查询batchs，其中*adatas为星号解包，即将列表adatas的元素展开传入：
# [adata_ref, *adatas] = [adata_ref, batch1, batch2, batch3]
# join="outer"代表所有基因取并集，毕竟可能基因不完全一致，如果没有就补 0/NA
adata_concat = anndata.concat([adata_ref, *adatas], label="batch", join="outer")

# 依旧统一类别顺序
adata_concat.obs["celltype"] = (
    adata_concat.obs["celltype"].astype("category").cat.reorder_categories(adata_ref.obs["cellty
")

# 依旧统一配色为ref的
adata_concat.uns["celltype_colors"] = adata_ref.uns["celltype_colors"]

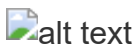
# 依旧画图
sc.pl.umap(adata_concat, color=["batch", "celltype"])
```



- 左图
 - 现象：4个batch在每个cluster中都混合了
 - 结论：整合成功，聚类由细胞类型驱动，而非batch
- 右图
 - 现象：每个cluster对应单一细胞类型
 - 结论：生物差异清晰，细胞得以清晰分离
- 隐患
 - celltype里，batch0是原注释，但查询batch的是ingest预测，因此需要验证 ingest vs celltype_orig

```
adata_query = adata_concat[adata_concat.obs["batch"].isin(["1", "2", "3"])].copy()

sc.pl.umap(adata_query, color=["batch", "celltype", "celltype_orig"], wspace=0.4)
```



该图不宜阅读，可以换成混淆矩阵：

```
# 筛选类型：取出ingest和真值的类型交集
conserved_categories = adata_query.obs["celltype"].cat.categories.intersection(
    adata_query.obs["celltype_orig"].cat.categories
)

# 筛选细胞：取出仅属于类型交集的细胞
obs_query_conserved = adata_query.obs.loc[
    adata_query.obs["celltype"].isin(conserved_categories) & adata_query.obs["celltype_orig"].isin(
        conserved_categories
    )
].copy()

# 删除不要的类别并统一类别顺序
obs_query_conserved["celltype"] = obs_query_conserved["celltype"].cat.remove_unused_categories()
obs_query_conserved["celltype_orig"] = (
    obs_query_conserved["celltype_orig"]
    .cat.remove_unused_categories()
    .cat.reorder_categories(obs_query_conserved["celltype"].cat.categories)
)

# 评估ingest准确度
pd.crosstab(obs_query_conserved["celltype"], obs_query_conserved["celltype_orig"])
```

celltype_orig	alpha	beta	ductal	acinar	delta	gamma	endothelial
alpha	1811	3	13	1	1	20	0
beta	56	806	6	1	10	37	0
ductal	7	5	684	241	0	0	0
acinar	2	3	3	165	0	3	0
delta	6	3	2	0	305	72	0
gamma	1	5	0	1	0	184	0
endothelial	2	0	0	0	0	0	36

- 对角线：预测正确的数字
- 非对角线：预测错的数字

也可以使用以下代码，在不取celltype交集的同时查看细胞类别混淆矩阵：

```
# 不是adata_query_conserved
pd.crosstab(adata_query.obs["celltype"], adata_query.obs["celltype_orig"])
```

Harmony



可视化各批次的分布

为了让UMAP里清晰观察批次分布，可以将其他batch变灰色

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for batch, ax in zip(["1", "2", "3"], axes, strict=True):
    sc.pl.umap(adata_concat, color="batch", groups=[batch], ax=ax, show=False)
```



此外，除了使用UMAP可视化各批次的分布，还可以使用密度图

```
sc.tl.embedding_density(adata_concat, groupby="batch")
```

```
sc.pl.embedding_density(adata_concat, groupby="batch")
```

 alt text